

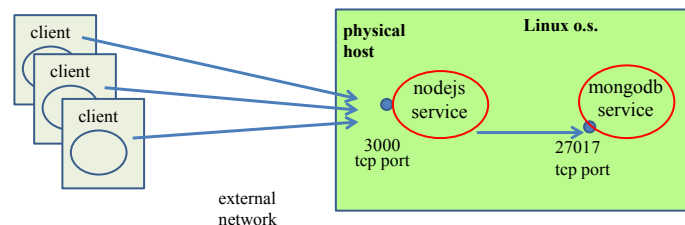
Seminario su docker e docker compose per progetti di ASW

Prof. Vittorio Ghini - 30/10/2024

L'esempio di applicazione da realizzare.

Vediamo passo per passo come costruire una applicazione web, implementata mediante **nodejs**, che sfrutta le librerie **express** per interfacciarsi un database gestito dal dbms **mongodb**.

Struttura dell'applicazione



Cosa fa l'applicazione:

Il server web nodejs accetta 3 tipi di richieste http:

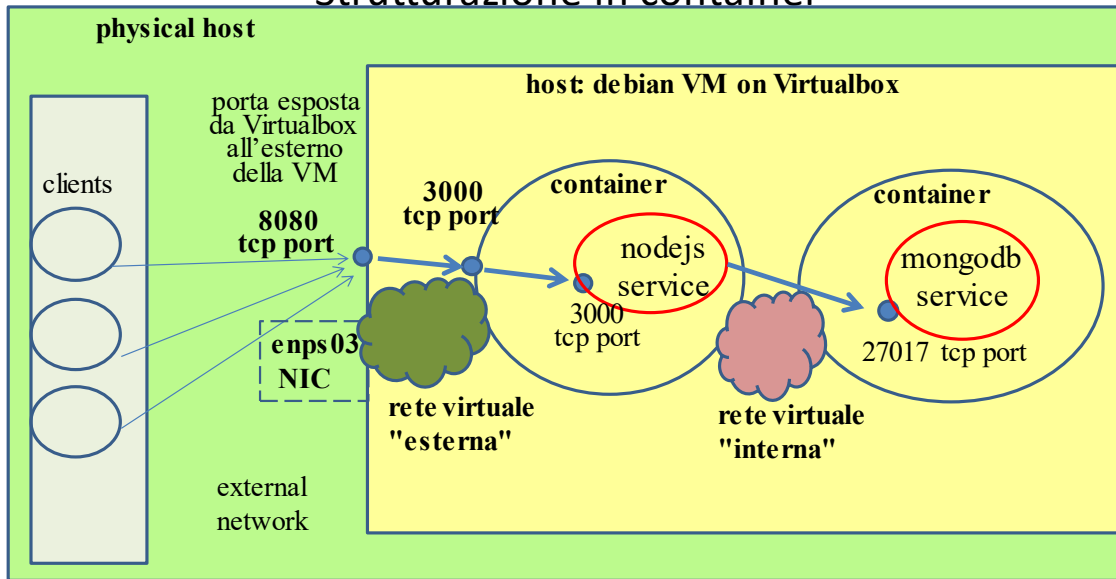
- di tipo **POST** per il percorso **/submit** contenente due parametri seq1 e seq2, poi calcola due nuove stringhe, funzione dei due parametri, salva nel database la quadrupla formata dalle due stringhe vecchie e dalle due calcolate, infine restituisce al client la quadrupla.
- di tipo **GET** per il percorso **/show** e restituisce al client il contenuto del database.
- di tipo **GET** per il percorso **/** che restituisce al client una pagina HTML con una form in cui inserire due stringhe seq1 e seq2. Un pulsante "Submit" manda queste due stringhe al web server mediante un POST sulla pagina /submit affinché vengano inserite nel database. La richiesta alla POST (la quadrupla) viene visualizzata dal browser. Questa pagina viene gestita con express, mongoose, vue, axios ma è primitiva, va migliorata. Mettetela a posto e poi mandatemi la correzione, grazie 😊 .

Struttura in container dell'applicazione:

Ciascuno dei due componenti, uno nodejs con express e l'altro mongodb, viene realizzato e pacchettizzato in un proprio **container docker**.

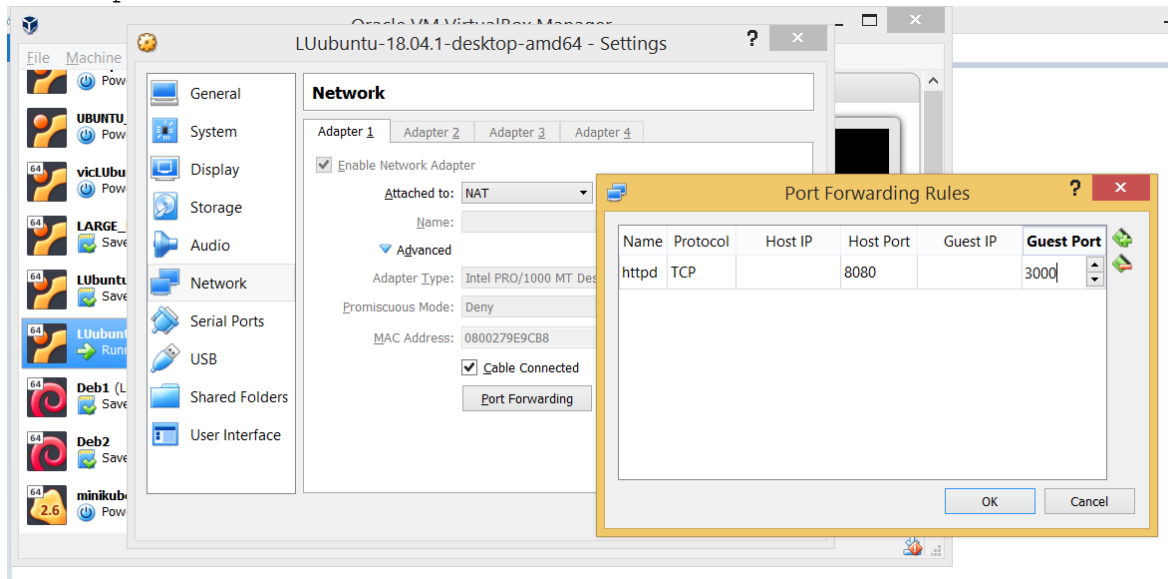
L'applicazione nel suo complesso (formata dai due container) viene costruita ed eseguita in una macchina Linux, sfruttando i pacchetti docker e docker-compose.

Strutturazione in container



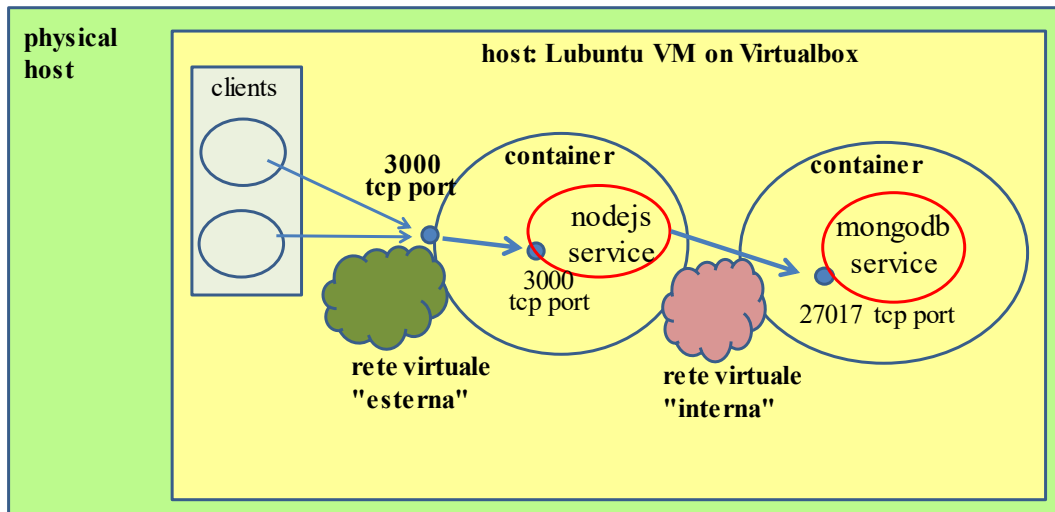
La struttura dell'applicazione, e la sua organizzazione in container è rappresentata nello schema seguente, in cui è indicato il caso in cui l'applicazione è eseguita su un o.s. Linux ospitato all'interno di una macchina virtuale (gialla) gestita dall'hypervisor virtualbox; nello schema compaiono anche i client web (browser), che non consideriamo parte dell'applicazione. Ovviamente, nulla vieta che l'applicazione venga eseguita direttamente su una macchina fisica con o.s. Linux.

Qualora l'applicazione venga eseguita all'interno di una macchina virtuale in Virtualbox, occorrerà configurare virtualbox affinché renda visibile all'esterno della macchina virtuale la porta su cui si vuole ricevere le connessioni provenienti dai client. Si veda a tale proposito la figura che descrive come configurare il port forwarding in virtualbox, esponendo la porta 3000, pubblicata dal container, sulla porta esterna 8080 della VM.



Nulla vieta che i test vengano eseguiti collocando i clients all'interno della macchina virtuale stessa, come da schema nella seguente figura.

Strutturazione in container test con client collocati nella macchina virtuale



In questo caso, non occorrerà più configurare Virtualbox per esporre la porta del servizio fuori dalla macchina virtuale stessa; i client si collegheranno alla porta esposta esternamente al container di nodejs, usando 0.0.0.0 o localhost.

Nota bene: il codice sorgente dell'applicazione e tutti i file che servono a creare i container dell'applicazione e a metterli in esecuzione (in pratica tutti i file citati in questo documento) sono contenuti nell'archivio zippato ASWl4.zip

Organizzazione del documento

Questo documento è organizzato in 10 parti:

- 1) installazione dei pacchetti docker e docker-compose in una macchina Linux.
- 2) Costruzione della rete virtuale "interna" che connette tra loro i due container del web server nodejs e di mongodb e costruzione della rete virtuale "esterna" che fa comunicare il container del web server con la VM host.
- 3) Perché occorre modificare l'immagine di partenza dell'applicativo mongodb, rimuovendo i volumi anonimi preimpostati (/data/db e /data/configdb) e montando, su una directory specifica del container di mongodb, uno script che inizializza il database.

4) **Costruzione dell'immagine del container con mongodb e il database.**

Vedremo come realizzare il container con il dbms mongodb che mantiene e gestisce il database dba con una collection 'alignments'. Definiremo un file Dockerfile che consente di creare (build) l'immagine del container con mongodb ed il nostro database.

5) **Esecuzione del container con mongodb ed il database dba.**

Indicheremo poi come effettuare manualmente il dispiegamento e l'esecuzione del container ed il controllo del log del container in caso di errori.

6) **Costruzione ed esecuzione del container con dentro l'applicazione web basata su nodejs** ed express, con particolare accento sul modo di garantire che il servizio nodejs riesca collegarsi al servizio del container mongodb. Ciò sarà garantito collocando entrambi i container in una rete virtuale, precedentemente create, che chiameremo "interna" che automaticamente realizza un DNS implicito di docker. In tal modo, il nome del container di mongodb)o del servizio, usando docker compose) potrà essere usato come nome dell'host virtuale in cui mongodb è in esecuzione e potrà quindi essere risolto in un indirizzo IP mediante il quale nodejs si collega a mongodb.

7) **Esecuzione del container con dentro l'applicazione web** basata su nodejs, express e mongoose ed il controllo del log del container in caso di errori.

8) **Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i servizi e dei loro container.**

Armonizzazione ed Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i container necessari all'applicazione e dell'ambiente di esecuzione dei container, ed infine terminazione dell'applicazione, mediante **docker-compose**. Controllo del log dei servizi in caso di errori mediante docker-compose.

9) **Salvataggio e Pacchettizzazione dell'applicativo con dentro le immagini dei container.**

Indica come consegnare il progetto di ASW, usando più spazio disco, allo scopo di non dover costringere i docenti a rifare il build delle immagini.

10) **Riassunto comandi per gestire l'applicazione con docker compose.**

1) Installazione dei pacchetti docker e docker-compose in una macchina Linux.

1.1) Installazione di docker in una macchina ubuntu mediante script di convenienza.

```
# per installare docker occorre essere tra i sudoers

# installare curl
sudo apt-get update
sudo apt-get install curl

# installare docker engine - prendiamo quello originale di docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# verifica che docker sia installato
sudo docker -v
# una versione recente installata potrebbe essere questa:
# Docker version 27.3.1, build cel2230

# Le versioni recenti di docker comprendono già il sotto comando
# docker compose      (senza il trattino di unione)
# Verificare se esiste il sottocomando eseguendo:
docker compose version
# un output di una versione recente di docker potrebbe essere questo:
# Docker Compose version v2.29.7

# Le versioni recenti di docker contengono il comando docker buildx
# che estende il sotto comando docker build
# Verificare se esiste il sottocomando eseguendo:
docker buildx version
# un output di una versione recente di docker potrebbe essere questo:
# github.com/docker/buildx v0.17.1 257815°
```

1.2) Come eseguire il comando docker senza dover usare sudo.

```
# chi e' l'utente corrente nel gruppo docker
echo ${USER}
# inserire utente corrente nel gruppo docker
sudo usermod -aG docker ${USER}
# verificare che utente corrente è nel gruppo docker
id -nG
# ricaricare utente
su - ${USER}

# verifica che docker sia eseguibile senza sudo
docker -v
```

1.3) Cos'è il registry di docker

Il registry locale di docker è un servizio installato quando viene installato docker. Mantiene nel filesystem le immagini dei container che sono state utilizzate.

2) Costruzione della rete virtuale "interna" che connette tra loro i due container del web server nodejs e di mongodb e costruzione della rete virtuale "esterna" che fa comunicare il container del web server con la VM host.

Creo una rete virtuale che verrà usata per comunicare tra i container e la chiamo "interna". Lo scopo vero è non dover definire degli indirizzi IP per i container e sfruttare invece i nomi dei container come se fossero dei nomi di host, che verranno risolti in indirizzi IP dal DNS implicito che docker realizza automaticamente per ciascuna rete virtuale che crea. I container che usano una stessa rete virtuale possono vedere gli altri container di quella rete usando i nomi dei container come se fossero dei nomi di host, ed il DNS implicito li risolve. Creo la rete virtuale usando il comando:

```
docker network create -d bridge --internal interna
```

Verificare l'esistenza della rete con il comando

```
docker network ls
```

E, per vedere più dettagli sulla rete denominata "interna", eseguire:

```
docker network inspect interna
```

Creo la rete virtuale "esterna" che connette il web server all'host, usando il comando:

```
docker network create -d bridge esterna
```

3) Perché occorre modificare l'immagine di default dell'applicativo mongodb.

Il tipico container di mongodb inserisce la propria configurazione ed i file in cui memorizza i dati dei suoi database, in due sue directory `/data/db` e `/data/configdb`

Però, poiché il container mongodb potrebbe essere terminato volutamente o per crash, con la terminazione si perderebbero le modifiche apportate nel frattempo alla base di dati, perché non si salverebbero le modifiche.

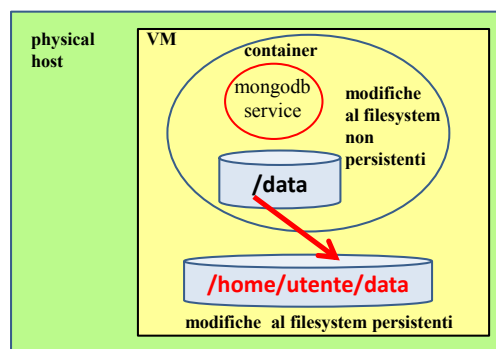
Per tale motivo, il container mongodb è fatto per mappare due proprie directory interne `/data/db` e `/data/configdb` in due directory esterne al container, cioè due directory della macchina host (fisica o virtuale) per assicurare la persistenza. In questo modo, se anche il container termina, il database rimane nel disco fisico (o virtuale).

La directory esterna può essere di tre tipi:

- a) **Bind:** è una directory definita esplicitamente dall'utente mentre crea il container (opzione `-v` di docker run, il primo argomento è la directory sull'host, ad esempio `-v /home/vic/data:/data`).
- b) **Named volume:** è una specie di directory creata dal daemon docker e collocata in una directory dell'host gestita dal daemon docker. Sopravvive alla fine del container. Poiché ne è noto il nome, può essere assegnata a più container. L'opzione `--mount` specifica per primo il nome del volume per primo (`--mount source=nomevolume,target=/percorso/assoluto/nel/filesystem/del/container`). Se il volume non esiste già allora viene creato.
- c) **Anonymous volume:** E' l'impostazione predefinita. In mancanza di impostazioni dell'utente, è il daemon docker che crea un volume senza nome (anonimo) e lo colloca in una directory dell'host gestita dal daemon docker. Ogni volta che viene eseguito un nuovo container viene creato un nuovo volume anonimo. Equivale all'opzione `-v` di docker run in cui non si mette il primo argomento (`-v =/percorso/assoluto/nel/container`).

In tutti i 3 casi, il database è salvato sul filesystem dell'host.

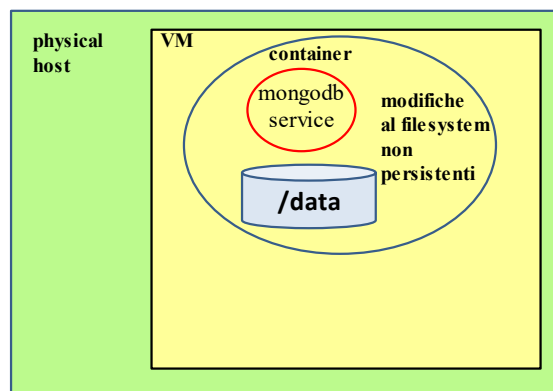
Rendere persistenti le modifiche al DB
salvando in volume esterno la
configurazione ed i dati



Purtroppo, questa impostazione di mongodb, che salva il database sul filesystem esterno (cioè dell'host), non va bene per consegnare il progetto di ASW.

Per consegnare il progetto di ASW, **invece**, occorre far sì che il container mongo non abbia bisogno di una directory esterna, che rimarrebbe nel vostro disco, ma scriva al proprio interno i dati. Occorre perciò configurare il container affinché **quella directory /data/db sia effettivamente una directory DEL filesystem proprio del container**. Occorre però salvare l'immagine del container dopo che avrete riempito il database con i dati necessari.

Non appoggiarsi su volume esterno.
**Occorre salvare l'immagine del container
per salvare le modifiche del DB**



Un ulteriore problema è che bisogna impedire che durante la creazione dei container vengano creati dei volumi anonimi per /data, che non sono utilizzati ma che comunque portano via spazio nel disco dell'host.

Nel Dockerfile con cui si crea l'immagine di un container, il comando VOLUME inserisce nell'immagine del container l'ordine di creare il volume nel momento in cui si esegue il container. L'immagine di mongo contiene questo ordine.

Purtroppo, non esiste un modo semplice per **rimuovere**, dall'immagine di un container, **l'ordine di creare un volume anonimo sull'host**. Per ottenere il risultato occorre creare un container e poi salvarne lo stato in una nuova immagine (comando export) senza salvare i volumi.

4) Costruzione dell'immagine del container con mongodb ed il database.

Il codice contiene una directory ASWl4 e due sottodirectory nodejs e mongodb in cui ci sono i file per costruire il container rispettivamente per l'applicazione web e per il database documentale.

4.1) Come l'immagine base del container mongodb consente di inizializzare il database stesso.

Su dockerhub https://hub.docker.com/_/mongo troviamo le informazioni sull'immagine del container base per mongo db. Nella sezione "Initializing a fresh instance" capiamo come può essere inizializzata l'istanza del database la prima volta che, nel build dell'immagine, lanciamo in esecuzione il demone mongod. E' infatti detto: "When a container is started for the first time it will execute files with extensions .sh and .js that are found in /docker-entrypoint-initdb.d. Files will be executed in alphabetical order. .js files will be executed by mongosh (mongo on versions below 6) using the database specified by the MONGO_INITDB_DATABASE variable, if it is present, or test otherwise. You may also switch databases within the .js script."

Ora la versione latest di mongo è mongo:8.0.1-noble, cioè la versione 8.0.1 basata su Ubuntu:noble.

L'inizializzazione del database la fa lo script docker-entrypoint.sh, contenuto nella directory /usr/local/bin del filesystem dell'immagine di mongo, il quale prende come argomento il nome del daemon mongod, contenuto nella directory /usr/bin, e lo avvia. Entrambe le directory sono contenute nella PATH di sistema del container e quindi lo script e l'eseguibile possono essere lanciati senza specificare i percorsi assoluti. Il daemon mongod a sua volta esegue gli script contenuti nella directory /docker-entrypoint-initdb.d scrivendo configurazione e dati del database nelle directory /data/db e /data/configdb (ma quest'ultima viene usata solo in caso di db replicato in cluster). Quindi, **se si crea una nuova immagine di container partendo da mongo e si vuole che i container originati da questa nuova immagine siano in grado di inizializzare il database nel modo sopra descritto, è necessario che all'inizio dell'esecuzione del container questo esegua lo script docker-entrypoint.sh con argomento mongod.**

Questa esigenza viene soddisfatta se la nuova immagine viene creata con un **Dockerfile** che ha almeno i seguenti comandi:

```
FROM mongo:8.0.1-noble
...
ENTRYPOINT ["docker-entrypoint.sh"]
EXPOSE 27017
CMD ["mongod"]
```

4.2) TRUCCO: Creare una nuova immagine base del container mongodb la quale NON crei i volumi anonimi /data/db e /data/configdb e che consenta l'inizializzazione consueta del database mediante script.

Vado nella directory ./ASWl4.

La nuova immagine "**mongonovolume**" viene creata eseguendo le seguenti operazioni, che trovate implementate nello script

FORCE_CREATE_NOVOLUME_BASE_IMAGE.sh nella directory ./ASWl4:

- **eseguire un container partendo dall'immagine base mongo:8.0.1-noble.**
- scoprire gli identificatori dei volumi anonimi creati dal container, per poterli rimuovere dopo la terminazione del container.
- **stappare il container ma senza rimuoverlo.**
- **esportare in un file lo stato del container stoppato** (non vengono esportati i volumi montati).
- **importare lo stato del container come nuova immagine nel registry locale, assegnandole il nome desiderato.**
- pulire il sistema eliminando file e oggetti non più necessari
- eliminando il file esportato,
- rimuovendo il container stoppato,
- eliminando i volumi creati.

In tal modo mi ritrovo nel registry locale una immagine, denominata **mongonovolume** che può essere successivamente usata come base per costruire una ulteriore nuova immagine di **mongodb** customizzata a mio piacere mediante scripts, che chiameremo mymongo.

Il Dockerfile per costruire la ulteriore nuova immagine mymongo dovrà contenere almeno questi comandi:

```
FROM mongonovolume:8.0.1-noble
...
ENTRYPOINT ["docker-entrypoint.sh"]
EXPOSE 27017
CMD ["mongod"]
```

Qui di seguito, lo script **FORCE_CREATE_NOVOLUME_BASE_IMAGE.sh** che serve a creare la nuova immagine mongonovolume.

Attenzione: ribadisco che questa immagine mongonovolume non fa partire il daemon mongod quando creiamo il container. Per avere una immagine che faccia partire il daemon mongod occorre crearne una nuova aggiungendo nel Dockerfile i comandi ENTRYPOINT e CMD sopra indicati.

Oppure, come vedremo più avanti, occorre eseguire il container mediante un docker-compose.yml file che specifica i comandi da eseguire all'avvio del container, ad esempio:

```
command: [ "docker-entrypoint.sh", "mongod" ]
```

```
# script FORCE_CREATE_NOVOLUME_BASE_IMAGE.sh
```

```
MONGOOriginalImage="mongo:8.0.1-noble"
```

```
MONGOContainerName="mongodb"
```

```
NoVolumeMONGOImageName="mongonovolume"
```

```
# se l'immagine e' gia' nel registry la elimino e la ricreo
```

```
FOUND=`docker image ls -q ${NoVolumeMONGOImageName}`
```

```
if [[ -n ${FOUND} ]] ; then
```

```
    echo "image ${NoVolumeMONGOImageName} esiste gia', LA ELIMINO E RICREO";
```

```
    docker rmi -f ${FOUND}
```

```
fi
```

```
# running container ${MONGOContainerName}
```

```
docker run -itd --name ${MONGOContainerName} ${MONGOOriginalImage}
```

```
# wait until container is up
```

```
NUMATTEMPTS=0
```

```
while OUT=`docker ps --no-trunc --filter "name=^/${MONGOContainerName}" --filter  
"status=running" --format "{{.ID}}"; [[ -z ${OUT} ]] ; do ((${NUMATTEMPTS}++)); echo  
${NUMATTEMPTS} sleep 1; done
```

```
# find anonymous volumes
```

```
VOLUMIseparatidavirgole=`docker ps --no-trunc --filter "name=^/${MONGOContainerName}" --  
filter "status=running" --format "{{.Mounts}}"
```

```
# stopping container ${MONGOContainerName}
```

```
docker stop ${MONGOContainerName}
```

```
# export stopped container
```

```
docker export ${MONGOContainerName} > ${MONGOContainerName}_export.tar
```

```
# import image in local docker registry
```

```
docker import ${MONGOContainerName}_export.tar ${NoVolumeMONGOImageName}
```

```
rm ${MONGOContainerName}_export.tar
```

```
# finito, faccio pulizia di cio' che non serve piu'.
```

```
# remove stopped container
```

```
docker rm ${MONGOContainerName}
```

```
# remove anonymous volumes
```

```
if [[ -n ${VOLUMIseparatidavirgole} ]] ; then
```

```
    VOLUMIseparatidaspazi=${VOLUMIseparatidavirgole//,/ }
```

```
    docker volume rm -f ${VOLUMIseparatidaspazi}
```

```
fi
```

```
# lists new mongo image
```

```
docker images ${NoVolumeMONGOImageName}
```

4.2) Scrivere i file necessari ad inizializzare il db mongo.

Sono nella directory ./ASW14/mongodb. **Predisponiamo uno script da far eseguire all'interno del container mongo senza volumi la prima volta che esegue, nel nostro caso durante il build. Lo script inizializza il nostro database dentro mongo, creando un database che chiamo "dbsa" una collection che chiamo "alignments" e inserendo qualche documento nella collection.**

Creo e scrivo un file in linguaggio javascript che farò eseguire dal mio mongodb solo la prima volta che lo metterò in esecuzione. Tutti i file di scripting .js o .sh che, nel container, si trovano nella directory /docker-entrypoint-initdb.d/ verranno eseguiti la prima volta che il container viene messo in esecuzione, o meglio, la volta che il container viene messo in esecuzione e nel db non ci sono database.

Chiamo il file "**mydbinit.js**" e questo è il contenuto:

```
// mydbinit.js script per inizializzare e popolare il database.
var conn = new Mongo();
var db = conn.getDB('dbsa');

// crea collection 'alignments' e la lascia se già esiste
db.createCollection('alignments', function(err, collection) {});

// elimina gli eventuali documenti della collection 'alignments'
// nel caso esistesse già
try { db.alignments.deleteMany( { } ); } catch (e) { print (e); }

db.alignments.insert({\"s1\": \"GCATGCU\", \"s2\": \"GATTACA\",
\"as1\": \"GCATGC-U\", \"as2\": \"G-ATTACA\"})
```

Poi assegno i permessi di esecuzione a tutti per gli script:

```
chmod 777 mydbinit.js docker-entrypoint.sh
```

Costruisco il Dockerfile che costruisce l'immagine definitiva dei miei container, partendo dall'immagine senza volumi e mettendo gli script di inizializzazione dove servono.

Nel Dockerfile parto usando l'immagine di mongo senza volumi

```
FROM mongonovolume
```

Inserisco nel Dockerfile quel che serve per far eseguire l'inizializzazione del DB alla prima esecuzione del container, nel nostro caso durante il build.

Per prima cosa aggiungo una riga in cui si ordina di copiare il file mydbinit.js nella directory /docker-entrypoint-initdb.d/ del container in cui mongodb cercherà gli script da eseguire dopo che il container è stato messo in esecuzione.

```
COPY ./mydbinit.js /docker-entrypoint-initdb.d/
```

Aggiungo queste due righe per settare i permessi di esecuzione

```
WORKDIR /usr/local/bin/  
RUN chmod 777 /docker-entrypoint-initdb.d/mydbinit.js
```

A fini di documentazione dico che il container usa le porte 27017, 27018 e 27019, quelle di default.

```
EXPOSE 27017 27018 27019
```

Stabilisco quale eseguibile eseguire una volta che il container è in esecuzione e quali argomenti di default gli vengono passati. Visto che il percorso di script ed eseguibili dovrebbero essere già nella PATH del container, dovrebbe essere sufficiente scrivere:

```
ENTRYPOINT ["docker-entrypoint.sh"]  
CMD ["mongod"]
```

In conclusione, il Dockerfile per la nostra immagine di mongo dovrebbe essere questo:

Dockerfile per creare l'immagine del nostro container mongo inizializzato.

```
FROM mongonovolume  
COPY ./mydbinit.js /docker-entrypoint-initdb.d/  
WORKDIR /usr/local/bin/  
RUN chmod 777 /docker-entrypoint-initdb.d/mydbinit.js  
EXPOSE 27017 27018 27019  
ENTRYPOINT ["docker-entrypoint.sh"]  
CMD ["mongod"]
```

4.5) Effettuare il build dell'immagine del container.

Abbiamo i file che ci servono, ora facciamo il build della nostra immagine di mongo che avrà il nostro db inizializzato e popolato.

Creare l'immagine nuova di mongo, che chiamo "mymongo" lanciando il build che usa il nuovo Dockerfile:

```
docker build -t "mymongo" . NB: C'è un punto alla fine
```

Se tutto va a buon fine, vedrò tra le immagini del registry locale anche la mia mymongo, lanciando il comando:

```
docker images | grep mymongo
```

Quella è l'immagine del container creato e salvato in locale.

5) Esecuzione del container con mongodb ed il database dba

Verifico che la rete virtuale che ho chiamato "interna", e che servirà a connettere i due container, sia ancora in esecuzione, eseguendo il comando:

```
docker network ls -f name=interna
```

mi aspetto un output di questo tipo

NETWORK ID	NAME	DRIVER	SCOPE
7e9f77130cd4	interna	bridge	local

Se per caso la rete non esistesse dovrei crearla, come indicato al precedente punto 2, eseguendo il comando:

```
docker network create -d bridge --internal interna
```

Creata la rete virtuale, **metto in esecuzione in background** il container contenente mongodb, partendo dall'immagine che ho creato, eseguendo:

```
docker run -itd --network interna -p 27017-27019:27017-27019  
--name mongodb mymongo
```

Notare che ho assegnato come nome "mongodb" al container che metto in esecuzione.

Notare che ho esposto le porte di mongodb sull'host per potermi collegare PER DEBUGGING con un applicativo che interroga il database. Normalmente non serve perché i nostri due container mongo e nodejs si affacciano alla stessa rete denominata "interna" e quindi vedono le rispettive porte di protocollo.

Verifico che il container battezzato mongodb sia in esecuzione, invocando il comando:

```
docker ps -a
```

Mi aspetto un output come segue:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
0f70e9a6f83e	mymongo	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes	
0.0.0.0:27017-27019->27017-27019/tcp		mongodb			

In caso di errori potreste vedere che lo status del container non è Up bensì è **Exited**. In tal caso, lanciate il comando seguente per visualizzare il contenuto del log del container chiamato mongodb e cercare di capire cosa è successo. Potete usarlo in generale per debugging anche mentre il container è ancora in esecuzione.

```
docker logs mongodb
```

Se invece il container è in esecuzione normalmente (status Up), è bene verificare che tutto sia funzionante; perciò **faccio eseguire, nel container di mongodb in esecuzione, un comando che usa la shell del client mongosh per eseguire uno script .js passato a riga di comando.** Questo script interroga mongodb chiedendo di visualizzare il contenuto della collection

```
docker exec -it mongodb /usr/bin/mongosh --eval "var conn = new  
Mongo(); var db = conn.getDB('dbsa'); var cursor =  
db.alignments.find(); while ( cursor.hasNext() ) { printjson(  
cursor.next() ); }"
```

Ci aspettiamo un output come quello che segue, che indica che nel nostro db dbsa c'è un solo documento:

```
MongoDB shell version v4.0.9  
connecting to:  
mongodb://127.0.0.1:27017/?gssapiServiceName=mongodb  
Implicit session: session { "id" : UUID("3df697d6-f2b8-46c5-a69d-  
43ea51d04b3d") }  
MongoDB server version: 4.0.9  
{  
  "_id" : ObjectId("5cd6a937518b4b8160497e25"),  
  "s1" : "GCATGCU",  
  "s2" : "GATTACA",  
  "as1" : "GCATGC-U",  
  "as2" : "G-ATTACA"  
}
```

5.1) Debugging del database.

5.1.1) Debugging stando dentro al container:

In caso di problemi sul database possiamo fare debugging entrando dentro al container con la shell di comandi di mongo e qui dentro dare dei comandi per verificare contenuto e comportamento del database.

```
docker exec -it mongodb /usr/bin/mongosh
```

Alcuni comandi utili per vedere il contenuto del db dbsa sono:

```
use dbsa  
db.alignments.find()  
exit
```

exit per terminare la shell /usr/bin/mongosh e uscire dal container.

5.1.2) Debugging stando all'esterno del container:

IN CASO DI PROBLEMI CON IL DATABASE, possiamo utilizzare il container mongodb **dall'esterno**, mediante applicativi che si connettono alla porta esposta dal container. ATTENZIONE, riesco a connettermi solo se ho esposto la porta 27917 e se ho connesso il mio container mongodb alla rete esterna, usando il comando: `docker network connect esterna mongodb`. Normalmente dovrei connettermi al database per tramite del web server oppure per tramite di docker exec come sopra spiegato.

Ad esempio, se nell'host locale abbiamo installato il client per connetterci a mongodb (sudo apt install mongodb-clients), allora possiamo connetterci al server mongodb, e in particolare al nostro database dbsa, aprendo come interfaccia la shell di comandi del client mongo, così:

```
mongo 127.0.0.1:27017/dbsa
```

Dall'interno di questa shell interattiva potremo lanciare comandi, ad esempio:

```
use dbsa
db.createCollection("alignments")
db.alignments.find()
db.alignments.insert({"s1": "GCATGCU", "s2": "GATTACA", "as1":
"GCATGC-U", "as2": "G-ATTACA"})
db.alignments.find()
exit
```

Oppure, sempre usando la shell di comandi del client mongo, potremmo direttamente eseguire delle query a riga di comando, passando delle query realizzate con degli script js, ad esempio inserendo dei nuovi documenti, così:

```
mongo 127.0.0.1:27017/dbsa --eval "var conn = new Mongo(); var db
= conn.getDB('dbsa'); db.alignments.insert({'s1': \"CAZCAZ\",
's2': \"ZACZAC\", 'as1': \"CAZCAZ-U\", 'as2': \"G-
ZACZACVAF\"});"
```

oppure visualizzando quelli esistenti, così:

```
mongo 127.0.0.1:27017/dbsa --eval "var conn = new Mongo(); var db
= conn.getDB('dbsa'); var cursor = db.alignments.find(); while (
cursor.hasNext() ) { printjson( cursor.next() ); }"
```

5.2) Terminazione e SALVATAGGIO dell'immagine del container e del suo database.

Il momento in cui decidete di terminare l'esecuzione del vostro container è critico, poiché se non salvate l'immagine del container perdete le modifiche che avete introdotto nel database. Perciò abbiamo due scelte:

5.2.1) o terminare il container ed eliminare il container stesso, perdendo le modifiche e rimanendo con la sola immagine base mymongo:

```
docker stop mongodb
docker rm mongodb
```

5.2.2) oppure terminare il container, salvarlo in una nuova immagine (con un nuovo nome, ad esempio mymongo.1) ed infine eliminare il container:

```
docker stop mongodb
docker commit -m "saved mongo" -a "autore" mongodb mymongo.1
docker rm mongodb
```

Salvando le modifiche, la volta successiva, per lanciare un container e ritrovare le modifiche dovreste usare come immagine la nuova mymongo.1 invece di mymongo, così:

```
docker run -d -p 27017-27019:27017-27019 --name mongodb mymongo.1
```


5.2.3) Oppure potete eliminare la vecchia immagine mymongo sostituendola con la nuova. Occorre stoppare il container, fare il commit della nuova immagine chiamandola mymongo.1, poi rimuovere il container stoppato (altrimenti non si può rimuovere la vecchia immagine), poi rimuovere la vecchia immagine, poi copiare la nuova immagine nella vecchia e infine rimuovere la nuova immagine.

Lo script commit_mongo_image.sh fa proprio queste operazioni:

```
docker stop mongodb
docker commit -m "saved mongo" -a "autore" mongodb mymongo.1
docker rm mongodb
docker rmi mymongo
docker tag mymongo.1 mymongo
docker rmi mymongo.1
```

6) Costruzione dell'immagine del container con dentro l'applicazione web basata su nodejs, express e mongoose

Il codice contiene una directory ASWl4 e due sottodirectory nodejs e mongodb in cui ci sono i file per costruire il container rispettivamente per l'applicazione web e per il database documentale.

Andare nella directory ./ASWl4/nodejs in cui sono presenti i file necessari.

```
./app
./app/package-lock.json
./app/package.json
./app/index.js
./app/src
./app/src/controllers
./app/src/controllers/controller.js
./app/src/lib
./app/src/lib/sequences_alignment.js
./app/src/routes
./app/src/routes/routes.js
./app/src/models
./app/src/models/alignmentModels.js
./Dockerfile
```

C'è un **Dockerfile** ed una sottodirectory app che contiene tre files index.js package.json e package-lock.json ed una sottodirectory src a sua volta contenente 4 sottodirectory: controllers, lib, models, routes. Le quattro sottodirectory contengono l'applicazione vera e propria.

Il file package.json contiene le dipendenze dell'applicazione, cioè le informazioni che servono a capire quali pacchetti per nodejs installare per poter eseguire correttamente tutte le funzioni richiamate nel codice javascript dell'applicazione.

Il file index.js è quello che deve essere eseguito per far partire l'applicazione vera e propria, mediante il comando:

```
node index.js
```

All'interno del file index.js c'è la parte che stabilisce la connessione tra il web service ed il dbms mongodb mediante la libreria express. Il file index.js è così fatto:

```
// file index.js

var express = require('express');
var bodyParser = require('body-parser');
var mongoose = require('mongoose')
var Alignment = require('./src/models/alignmentModels')

//Creo istanza di express (web server)
var app = express();

//importo parser per leggere i parametri passati in POST
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

//connessione al db
// il secondo mongodb, quello colorato in rosso, e' il nome del
// container di mongo o, usando docker-compose, il nome del servizio
// mongodb specificato nel compose file.

mongoose.connect('mongodb://mongodb/dbsa', { useNewUrlParser: true,
useFindAndModify: false });

//mongoose.connect('mongodb://username:password@host:port', {
//useNewUrlParser: true, useFindAndModify: false });

var routes = require('./src/routes/routes');
routes(app);

//metto in ascolto il web server
app.listen(3000, function () {
  console.log('Node API server started on port 3000!');
});
```

Notare che l'istruzione:

```
mongoose.connect('mongodb://mongodb/dbsa', { useNewUrlParser: true,
useFindAndModify: false });
```

usa il nome del container (mongodb, quello colorato in rosso) come se fosse un nome di host, poiché sappiamo che il container di mongodb lo collocheremo nella stessa rete virtuale chiamata "interna" del container dell'applicazione, e quindi il DNS implicito riuscirà a risolvere il nome del container nell'indirizzo IP del container all'interno della rete virtuale "interna".

Notare anche l'istruzione generica commentata

```
//mongoose.connect('mongodb://username:password@host:port', {
//useNewUrlParser: true, useFindAndModify: false });
```

che ricorda come sia possibile inserire anche uno username ed una password per accedere a mongodb. Nel nostro caso non è necessario poiché abbiamo configurato mongodb per non richiedere username e password.

Poi scriviamo (c'è già) il **Dockerfile** nella directory nodejs per costruire l'immagine del container nodejsapp dell'applicazione web.

```
FROM ubuntu:focal

ENV WORKINGDIR=/root/app
# indica dove verranno eseguiti i comandi indicati da RUN
WORKDIR ${WORKINGDIR}

RUN mkdir -p ${WORKINGDIR} && chmod 666 ${WORKINGDIR}

# copia tutta la directory app e le sottodirectory nel container
COPY ./app ${WORKINGDIR}/

# installo nel container i pacchetti necessari
# e poi butto gli archivi non più utili
RUN apt-get -y update && \
    apt-get -y install apt-utils && \
    apt-get -y install nodejs && \
    apt-get -y install npm && \
    apt-get -y clean
# installo nel container le dipendenze indicate in package.json
RUN npm install

# ricordo che devo rendere accessibile dall'esterno del container
# la porta 3000 su cui nodejs rimane in attesa di connessioni
# da parte dei client
# Occhio, non espongo la porta, questa EXPOSE è solo una nota per
# ricordare che devo esporre la porta quando eseguo il container.
EXPOSE 3000

# dico quale comando eseguire automaticamente quando eseguirò il
# container: serve per far partire il servizio web nel container.
CMD nodejs index.js
```

A questo punto è tutto pronto, posso costruire l'immagine del container con nodejs e la nostra applicazione, andando nella directory nodejs contenente il Dockerfile, ed eseguendo il comando:

```
docker build -t nodejsapp .
```

Il comando impiega un poco di tempo e poi finisce scrivendo su disco l'immagine di un container con nome nodejsapp.

Verificare l'esistenza con il comando:

```
docker images | grep nodejsapp
```

7) Esecuzione del container con dentro l'applicazione web basata su nodejs, express e mongoose

Ora possiamo fare partire il container con l'applicazione web, collegandolo alla rete interna affinché possa connettersi al database mongodb, usando il comando:

```
docker run -itd --rm --network interna --name nodejsapp -p 3000:3000 nodejsapp
```

Poi colleghiamo il container con il web server alla rete esterna affinché possa essere raggiunta dalle richieste http che arrivano sull'host, mediante il comando:

```
docker network connect esterna nodejsapp
```

Notare che attacco il container alla stessa rete virtuale **"interna"** del container di mongodb. Inoltre assegno nome **"nodejsapp"** al container che metto in esecuzione. Con il parametro **-p 3000:3000** dico che voglio che la porta 3000 del container sia accessibile anche da fuori il container (cioè dall'host), in modo che i client esterni possano accedere al server nodejs che lavora ed attende sulla porta 3000. Il parametro **--rm** significa che quando il container terminerà l'esecuzione, cioè verrà stoppato, docker dovrà eliminarlo automaticamente.

Per usufruire dell'applicazione, da dentro l'host fisico in cui eseguo il container, posso lanciare un browser grafico o testuale con cui mandare un messaggio HTTP POST con cui passo due parametri seq1 e seq2 e provo così la generazione di 2 stringhe, as1 ed as2, ed il successivo salvataggio in mongodb della quadrupla in formato json {s1,s2,as1,as2} dove s1=seq1 ed s2=seq2.

Ad esempio, lanciando il comando:

```
curl --header "Content-Type: application/x-www-form-urlencoded" --request POST --data 'seq1=ALFABETAGAMMA&seq2=VAFFA' 0.0.0.0:3000/submit
```

faccio generare due stringhe as1 ed as2 e viene restituito un documento json che è quello che è stato salvato nel database mongo.

```
{"s1":"ALFABETAGAMMA","s2":"VAFFAERIVAFFA","as1":"A-LF-ABETAGAMMA","as2":"VA-FFA-ERIVAFFA"}
```

Per visualizzare il contenuto del db, mando una richiesta GET a /show

```
curl http://0.0.0.0:3000/show
```

Per stoppare il container userò il comando:

```
docker stop <nome container>
```

cioè, nel caso specifico:

```
docker stop nodejsapp
```

Infine, per eliminare il container, dopo averlo stoppato, userò il comando:

```
docker rm <nome container>
```

cioè, nel caso specifico:

```
docker rm nodejsapp
```

Poiché questo container non salva nessun dato, non ha senso fare un commit dopo averlo utilizzato.

8) Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i container.

Per automatizzare tutto quanto, è necessaria una modifica del codice dell'applicazione nodejsapp perché per riuscire a connettersi a mongodb occorre aspettare che mongodb sia stato inizializzato.

Inserisco perciò una orrenda pausa di 10 secondi prima di tentare la connessione da nodejsapp a mongodb. Aggiungo inoltre un po' di codice per visualizzare il tipo di errore. Agisco perciò sul file index.js

Tratto dal nuovo file **index.js**

```
// aspetto 10 sec che il container di mongo sia su
function pausecomp(millis)
{
  var date = new Date();    var curDate = null;
  do { curDate = new Date(); } while(curDate-date < millis);
}
pausecomp(10000);

//connessione al db^M
mongoose.set('useFindAndModify', false);
mongoose.set('connectTimeoutMS', 30);
mongoose
  .connect(
    'mongodb://aswl_mongodb_1.aswl_interna:27017/dbsa',
    { useNewUrlParser: true })
  .then(() => console.log('MongoDB Connected'))
  .catch((err) => console.log(err));
```

Ora possiamo procedere ad automatizzare creazione ed esecuzione dei container.

Nella directory ASW, radice del mio progetto, **costruisco il file docker-compose.yml** che indica:

- come fare il build delle immagini dei due container,
- come creare la rete virtuale interna e quella esterna,
- come mettere in esecuzione tutti i container.
- come terminare l'esecuzione di tutti i container e rimuoverne le immagini e rimuovere anche le due reti virtuali create

Il file **docker-compose.yml** è fatto come segue:

```
# project name "aswl" è in var COMPOSE_PROJECT_NAME=aswl nel file .env

services:
# il servizio che deve partire per secondo
nodejsapp:
  build:
    context: ./nodejs
    dockerfile: Dockerfile
  image: nodejsapp
  command: [ "nodejs", "index.js" ]
  depends_on:
    - mongodb
# Questa porta DEVO pubblicarla per vederla dall'host e fuori
dall'host.
  ports:
    - 3000:3000
  networks:
    - interna
    - esterna
mongodb:
  build:
    context: ./mongodb
    dockerfile: Dockerfile
  image: mymongo
  command: [ "docker-entrypoint.sh", "mongod" ]
  networks:
    - interna

networks:
interna:
  driver: bridge
  internal: true
  driver_opts:
    com.docker.network.bridge.name: "br-interna00000"
esterna:
  driver: bridge
  internal: false
  driver_opts:
    com.docker.network.bridge.name: "br-esterna00000"

#volumes: non li metto, salvo nel filesystem del container mongo
```

Per costruire le immagini dei container, stando nella directory del file docker-compose.yml, eseguire il comando:

```
docker compose build
```

Per mettere in esecuzione tutti i container, creando le reti necessarie, stando nella directory del file docker-compose.yml, eseguire il comando:

```
docker compose up -d
```

Per terminare l'esecuzione di tutti i container, eseguire il comando:

```
docker compose stop
```

Per terminare l'esecuzione di tutti i container, ed anche eliminare i container e le reti create, ma mantenere le immagini, eseguire il comando:

```
docker compose down
```

Per terminare l'esecuzione di tutti i container, ed anche eliminare i container e le reti create, ed eliminare pure le immagini dei container creati, eseguire il comando:

```
docker compose down --rmi all
```

Notare che docker-compose assegna ai container e alle reti che crea dei nomi strani ma prevedibili. Infatti, se:

- la directory in cui si trova il file docker-compose.yml si chiama ASW14,
- un servizio da creare nel file si chiama mongodb,
- di quel servizio viene creata una sola replica,
- la rete l'ho chiamata "interna",

allora docker compose fa sì che:

- il container creato per il servizio si chiamerà **asw14-mongodb-1**,
- la rete si chiamerà **asw14-interna-1**,
- l'host virtuale in cui è contenuto il container si chiamerà come il servizio **mongodb**, ma anche come il container, **asw14-mongodb-1**, o anche **asw14-mongodb-1.asw14-interna-1**.

Nota Bene: però posso decidere io il nome del progetto (asw14) che costituisce il prefisso delle risorse create. Basta creare nella directory base del progetto, cioè quella che contiene il file docker-compose.yml, un file che si chiama **.env** e che contiene una riga che setta una variabile così:

```
COMPOSE_PROJECT_NAME=asw14
```

9) Salvataggio e Pacchettizzazione dell'applicativo con dentro le immagini dei container.

Come consegnare il progetto di ASW, fornendo il codice, il necessario per fare il build delle immagini e anche le immagini dei container già costruite? Ciò serve allo scopo di non dover costringere i prof. Mirri e Delnevo a rifare il build delle immagini dei container.

Andare nella directory base del progetto, ASWl4, quella che contiene il file docker-compose.yml ed eseguire la seguente sequenza di comandi:

Creo le immagini di tutti i container
docker compose build

Salvo in un file ciascuna immagine dei container che devo usare, mettendolo nella directory base del progetto.

```
docker save nodejsapp > nodejsapp_save.tar
docker save mymongo > mymongo_save.tar
```

Faccio un archivio gzipato della directory di progetto, le immagini dei container salvate e le sottodirectory. Poiché la mia directory di progetto si chiama ASWl4 faccio così:

```
cd ../
tar cvzf ASWl4_saved.tgz ASWl4/
```

COPIO il mio mega archivio ASWl4_saved.tgz in una chiavetta USB e da qui nella macchina Linux (fisica o virtuale) in cui voglio eseguire il mio applicativo.

Nella nuova macchina Linux, dove ho copiato l'archivio ASWl4_saved.tgz, e dove sono già installati docker e docker compose, eseguo questi comandi, iniziando con l'estrazione dei file.

```
tar xvzf ASWl4_saved.tgz
```

Vado nella directory di progetto appena creata.

```
cd ASWl4/
```

Carico nel registro locale le immagini dei container del mio progetto.

```
docker load < mymongo_save.tar
docker load < nodejsapp_save.tar
```

Verifico che le immagini dei container siano state caricate localmente.

docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nodejsapp	latest	32c71fac8cbc	7 hours ago	449MB
mymongo	latest	0e0253d1853b	12 hours ago	442MB

Lancio il mio applicativo senza fare build.
docker compose up -d

Fatto.

NB: Distinzione tra i comandi "docker save" e "docker export":

Il comando **docker save** salva l'immagine nodejsapp presente nel registro locale.

```
docker save nodejsapp > nodejsapp_save.tar
```

Invece, il comando **docker export** salva l'immagine del container di nome nodejsapp stoppato ma non ancora eliminato.

```
docker export nodejsapp > nodejsapp_export.tar
```

10) RIASSUNTO COMANDI: BUILD, RUN con docker compose, USO, **ESPORTAZIONE** e DISTRUZIONE.

```
# BUILD
  cd ./ASWl4
  ./mongodb/FORCE_CREATE_NOVOLUME_BASE_IMAGE.sh
  docker compose build

# RUN
  docker compose up -d

# USO applicazione mediante POST e GET.
# aggiungo sequenze al database con POST
  curl --header "Content-Type: application/x-www-form-urlencoded"
    --request POST --data 'seq1=LANCIA&seq2=DELTA'
    http://0.0.0.0:3000/submit
# guardo contenuto con GET su /show
  curl http://0.0.0.0:3000/show
# oppure posso popolare il db con uno script
  ./popola_db.sh

# SALVO STATO DEL CONTAINER mongodb mettendolo nella immagine mymongo
  docker compose stop
  docker commit -m "saved" -a "autore" aswl4-mongodb-1 mymongo.1
  docker compose down
  docker rmi mymongo
  docker tag mymongo.1 mymongo
  docker rmi mymongo.1

# PER CONSEGNARE: faccio un tar con immagini e progetto completo.
  docker save nodejsapp > nodejsapp_save.tar
  docker save mymongo > mymongo_save.tar
  docker compose down -rmi all
  cd ../
  tar cvzf ASWl4_saved.tgz ASWl4/
# CONSEGNO il tar file ASWl4_saved.tgz
-----
# PER UTILIZZARE IL PROGETTO RICEVUTO:
  tar xvzf ASWl4_saved.tgz
  cd ASWl4

# carico le immagini dai due archivi
  docker load < mymongo_save.tar
  docker load < nodejsapp_save.tar
# faccio partire l'applicazione senza fare il build delle immagini
  docker compose up -d
# verifico di avere il database già popolato
  ./show_db_from_nodejsapp.sh
# ok, elimino tutto
  docker compose down --rmi all
```

Esempio Script popola.sh

```
while read sequenza1 sequenza2 ; do
  if [[ -n ${sequenza1} && -n ${sequenza2} ]] ; then
    curl --header \
      "Content-Type: application/x-www-form-urlencoded"\
      --request POST\
      --data 'seq1='${sequenza1}'&seq2='${sequenza2}'\
      http://0.0.0.0:3000/
    echo " "
  fi
done < datainput.txt
```

Esempio file di input: datainput.txt

```
MelaCotogna BananaSplit
GattoMannaro CaneDiMerda
FIAT127Special LanciaDeltaHFIntegrale8v
MaremmaMaiala Spezzatino
FessiDigitali RadicchioAmaro
```

Esempio Script show_db.sh

```
curl --header "Content-Type: application/x-www-form-urlencoded"\
  --request GET http://0.0.0.0:3000/show
```

Comandi utili per durante sviluppo.

Durante lo sviluppo probabilmente dovrete modificare il container nodejsapp senza dover modificare mongodb.
E' possibile fermare, modificare e far ripartire il solo container nodejsapp

```
# fermo ed elimino il solo container nodejsapp, mongo rimane attivo
# ordinando di fermare il servizio
docker compose stop nodejsapp
docker compose rm -f nodejsapp
... faccio le modifiche al codice di nodejapp
docker compose build nodejsapp
docker compose up -d nodejsapp
```